Business Intelligence & Data Warehouse

Technical Report

Andre'D Antoni - 265984

Anthony Josephsen Arroyo - 265987

Bjørn Erik Østby – 885279

Lars Hovde Nordås - 265974

Mikkel Stensvold – 246426

Innholdsfortegnelse

1. AI Assistance Disclosure 3
2. Executive summary 4
3. Technical implementation overview 5
3.1 Brief documentation of warehouse design decisions 5
3.1.2 Purpose 5
3.1.3 Overall design 5
3.1.4 Grain 6
3.1.5 Dimensions 7
3.1.6 Key Design Choices 7
3.2 ETL Process Implementation 8
3.2.2 Transform dim_customer 8
3.2.3 Transform dim_seller 9
3.2.4 Transform dim_date 9
3.2.5 Transform dim_product 9
3.2.6 Transform fact_order_items 10
3.2.7 Transform fact_delivery 10
3.3 Data Quality Issues and Handling 11
4. Key Insights (Simple and Direct) 12
4.1 Sales 12
4.2 Geography 12
4.3 Deliveries 12
4.4 Customers 12
4.5 Forecast 12
5. Business recommendations 12
5.1 Improve Delivery Performance 12
5.2 Focus on High-Performing Product Categories 12
5.3 Strengthen Presence in Key States 12
5.4 Prepare for Seasonal Peaks 13
5.5 Maintain High Customer Satisfaction 13
6. Team member contributions 13
7. Bibliography 14

1. AI Assistance Disclosure

Parts of this project were developed with assistance from AI tools to support the technical implementation and understanding of analytical concepts. These tools were used only as support, not as replace for our own work.
We used ChatGPT (OpenAI, GPT-5 model) to help understand analytical concepts, refine SQL queries for the PostgreSQL data warehouse and clarify PowerBI functions, visualization, dashboard development and DAX logic.
We used Claude (Anthropic) Github Copilot in Visual Studio Code as support when writing code for the predictive analysis, mainly for code suggestions and improving the structure, also an interpretation of the model behaviour.

No AI generated text was used directly and all outputs from AI tools were reviewed, rewritten in our own words, tested and validated by the group to ensure correctness and academic integrity.

2. Executive summary

This project presents the design and implementation of a complete Business Intelligence and Data Warehouse solution based on the Olist e-commerce dataset. Our goal was to build a functioning data warehouse, create reliable ETL processes, and produce analytical dashboards that answer key business questions about sales performance, product trends, customer behavior, and delivery efficiency. Throughout the project, we focused on creating a model that is easy to understand, consistent, and directly connected to the decision-making needs of an online retail business.

The data warehouse uses a star schema with two fact tables, fact_order_items and fact_delivery, and four supporting dimensions: dim_customer, dim_product, dim_seller, and dim_date. We selected only the attributes necessary for analysis, which helped us keep the model clean and manageable. ETL processes were built in Pentaho Data Integration, where we loaded the datasets from the csv files, cleaned the data, handled duplicates, standardized text fields, and ensured that every fact row correctly matched the relevant dimension keys.

Using this warehouse, we developed several Power BI dashboards to explore business performance. The results show a clear upward trend in total sales over time, with noticeable seasonal peaks. A small number of product categories account for most of the revenue, especially “beleza_saude”, “relogios_presentes”, and “cama_mesa_banho”. The geographical analysis reveals that sales are heavily concentrated in a few states, particularly São Paulo, which stands out as the strongest market.

The delivery analysis highlights important operational challenges. Around half of the deliveries are late, and the average delivery time is approximately twelve days. Some regions consistently experience higher delay rates. However, customer review scores remain high, with an average above 4.0, suggesting that customers are generally satisfied despite the delivery issues.

Finally, the forecast visuals indicate that sales are expected to continue growing, with seasonal patterns repeating in future periods. This gives the business the opportunity to plan for high-demand months.

Overall, this project demonstrates how a structured BI and Data Warehouse approach can provide valuable insights into sales performance, logistics, and customer behaviour. The findings also support several practical business recommendations, such as improving delivery performance, strengthening presence in key states, and preparing inventory for seasonal peaks. As students, this project gave us hands-on experience with end-to-end data warehouse development, ETL processes, and analytical reporting using industry-relevant tools.

3. Technical implementation overview

3.1 Brief documentation of warehouse design decisions

3.1.2 Datasource

This project uses the Brazilian E-commerce Dataset (Olist) obtained from Kaggle (https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce). The dataset was published by Olist and contains real commercial data from the Brazilian e-commerce platform.

3.1.3 Purpose

The data warehouse is designed to support analysis of the Olist e-commerce dataset. We did not load every column from the raw source. Instead, we selected the attributes that are necessary to answer our business questions about orders, deliveries, products, customers and sellers. This reduces complexity and keeps the model focused

Business questions we target:

- How did the total sales revenue develop month by month?
- Which products categories and regions generate the highest sales?
- Which product categories sell the most in different regions, and how does this vary from month to month?
- Which regions have the highest share of delayed deliveries, and how does this affect the average delivery time?
- What is the average delivery time?
- What is the average order size per region?
- What percentage of deliveries are delayed per region?

3.1.4 Overall design

The data warehouse uses a star schema optimized for sales and delivery analysis. A star schema was chosen for its simplicity, fewer joins, and better performance compared to a snowflake design.
It includes four dimension tables:

- **dim_customer:** contains customer information such as location, unique ID, and coordinates.
- **dim_seller:** stores seller details such as ID, city, and state.
- **dim_product:** includes product identifiers and category names.
- **dim_date:** represents the time dimension with year, month, day, and weekday information.

Two fact tables record measurable business events:

- fact_order_items: sales transactions for each product line item.
- fact_delivery: delivery and review information for each completed order.

All tables are connected through surrogate key relationships for example: customer_key, seller_key, product_key, date_key.

3.1.5 Grain

Each fact table is defined at the lowest level of detail to support flexible aggregation:

- **fact_order_items**: One record per order line item, including item price, freight cost, and total value.
- **fact_delivery**: One record per order delivery, including delivery time in days, lateness flag, and review score/comment.

This ensures consistency and allows analysis at both detailed and summarized levels.

3.1.6 Keys and constraints

Each dimension table uses a surrogate primary key, defined as an auto-incrementing integer.
Fact tables reference these keys through foreign key constraints:

- customer_key - dim_customer
- seller_key - dim_seller
- product_key - dim_product
- date_key (and variations) - dim_date

These constraints enforce referential integrity, ensuring that all facts correspond to valid dimension entries and preventing orphaned records.

3.1.7 Indexing strategy

Primary keys automatically create clustered indexes on all dimension tables. Additional non-clustered indexes were added on the foreign key columns in the fact tables (customer_key, seller_key, product_key, date_key) and on the order_id column to speed up joins and filters during analytical queries.
This indexing strategy provides balanced performance between fast query execution and efficient ETL loading.

3.1.8 Dimensions

dim_customer

- Source: customer file.
- Attributes kept: customer_unique_id, customer_id, city, state.
- Why: dataset is anonymized; we only need location to group customers by region.

dim_product

- Attributes kept: product_id, product_category_name, weight, dimensions.
- Why: needed for category-level reporting and potential logistics metrics; other columns from raw weren't needed for our questions.

dim_seller

- Attributes kept: seller_id, city, state.
- Why: sellers can be analyzed by location; other seller attributes were not relevant.

dim_date

- Role: shared calendar for joining purchase date, estimated delivery date, delivered date, and order date.
- Attributes: full_date, year, month, month_name, day, weekday.
- Why: enables time series and drilldowns without recalculating dates in the BI tool.

3.1.9 Key Design Choices.

Subset of attributes:
We intentionally did not import every column from raw. Only columns that are referenced in the business questions (sales, products, locations, delivery times, reviews) were modeled. This keeps ETL simpler and the star schema smaller.

Surrogate keys in dimensions:
Dimensions use serial/surrogate keys (for example customer_key, product_key). Facts store these keys to avoid joining on long text IDs and to support slowly changing dimensions later if needed.

Separate facts for sales vs delivery:
Delivery info (estimated vs delivered) is not at the same grain as order items. Putting them in the same fact table would produce duplicates. Therefore we split:

- fact_order_items = commercial view
- fact_delivery = logistics/service view

Multiple date relationships in fact_delivery:
We kept purchase date, estimated delivery date and delivered date as separate foreign keys. This is necessary to calculate lead time and to mark late deliveries.

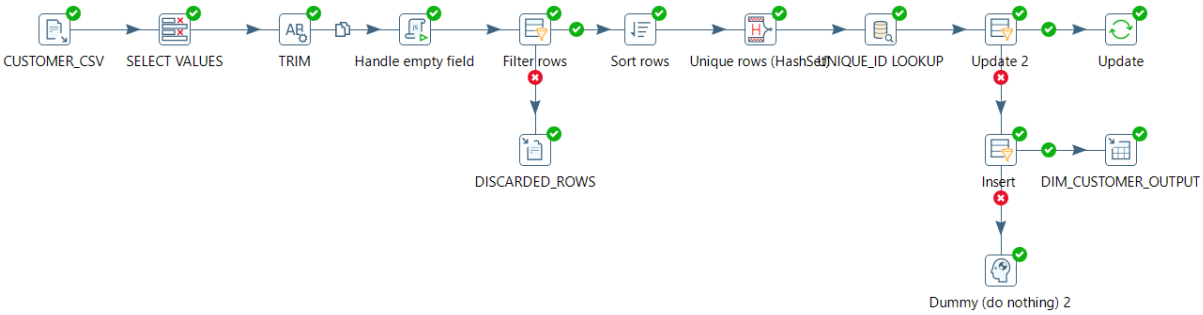
Handling of anonymous IDs:
Because customers and sellers are anonymized in the source, we only expose what is actually meaningful for analysis (city/state). Questions that require real names cannot be answered, so we did not model them.

3.2 ETL Process Implementation

All ETL workflows were implemented in Pentaho Data Integration (PDI). The process extracts, cleans, and loads data into the PostgreSQL data warehouse (dwh schema).

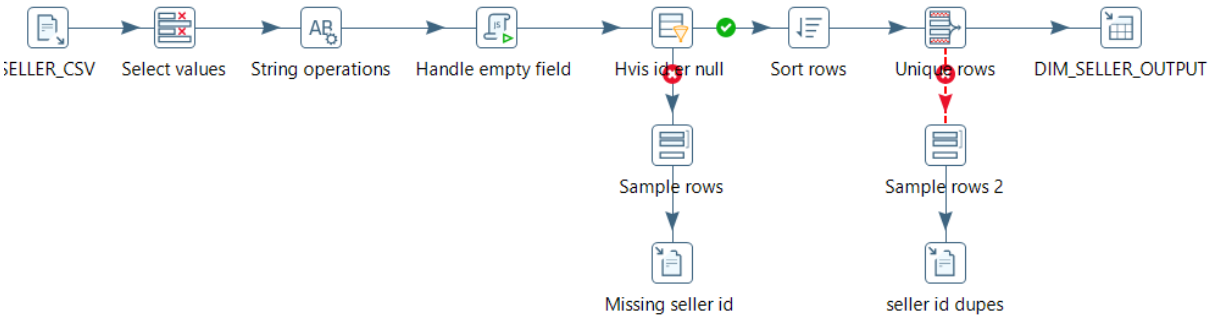
Each ETL flow applies cleaning and validation rules such as removing duplicates, handling missing values, standardizing text fields, and performing lookups to generate surrogate keys before loading the data into the dimensional model.

3.2.2 Transform dim_customer



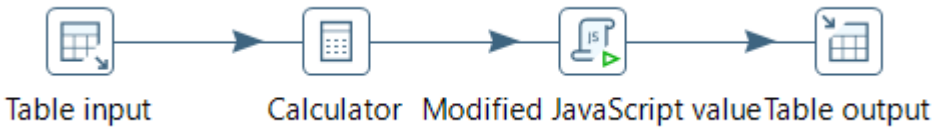
The dim_customer table is populated from olist_customers_dataset.csv, which contains customer identifiers, ZIP prefixes, city, and state. Only customer_unique_id, customer_city, and customer_state are retained, as the other fields do not support the analytical requirements of the warehouse. Because the raw dataset assigns multiple customer_id values to the same customer_unique_id, and the dimensional grain is defined as one row per unique customer, the ETL pipeline removes duplicates based on this. Text fields are trimmed and normalized, and missing or empty city/state values are replaced with “unknown” while invalid rows are written to a discard stream(based on missing unique_id). Before loading, the transformation performs a lookup against the existing dim_customer table to determine whether a customer already exists, matching records are updated, and new customers are inserted. The final cleaned records are loaded into dim_customer using a surrogate key generated during the insert/update step.

3.2.3 Transform dim_seller



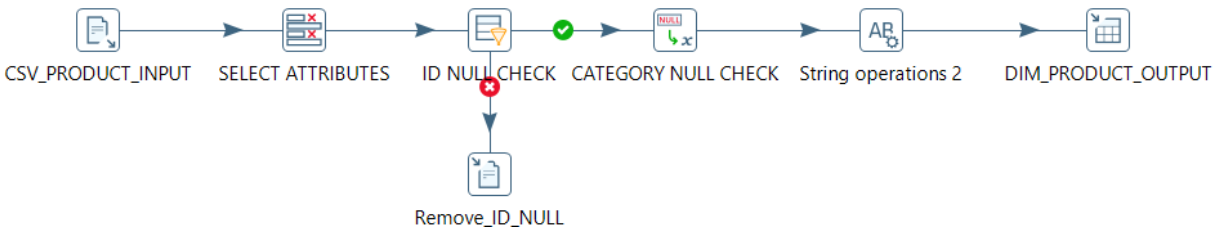
The dim_seller table is populated from olist_sellers_dataset.csv, which contains seller identifiers, ZIP prefixes, city, and state. Only seller_id, seller_city, and seller_state are kept, as the ZIP prefix offers no analytical value in the dimensional model. The **dimensional grain is set to one row per unique seller**, and the ETL process enforces this by removing duplicate seller_id entries and filtering out rows where the identifier is missing. City and state fields are trimmed and normalized, null or empty values are standardized to "Unknown," and the validated seller records are then loaded into dim_seller with a surrogate key during the insert/update step.

3.2.4 Transform dim_date



The dim_date table is generated using a simple ETL pipeline that creates a continuous range of calendar dates.

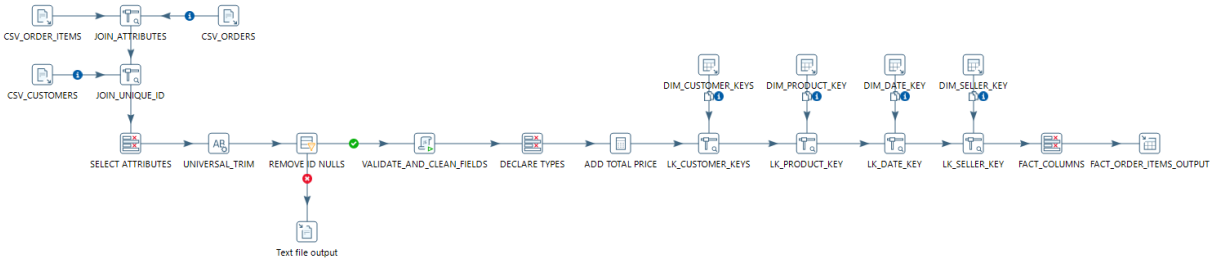
3.2.5 Transform dim_product



The dim_product table is populated from olist_products_dataset.csv, which contains product identifiers, category names, and various product attributes. Only product_id and product_category_name are retained in the dimension. The dimensional grain is defined as one row per unique product, and the ETL flow enforces this by first removing any rows where product_id is null, sending them to

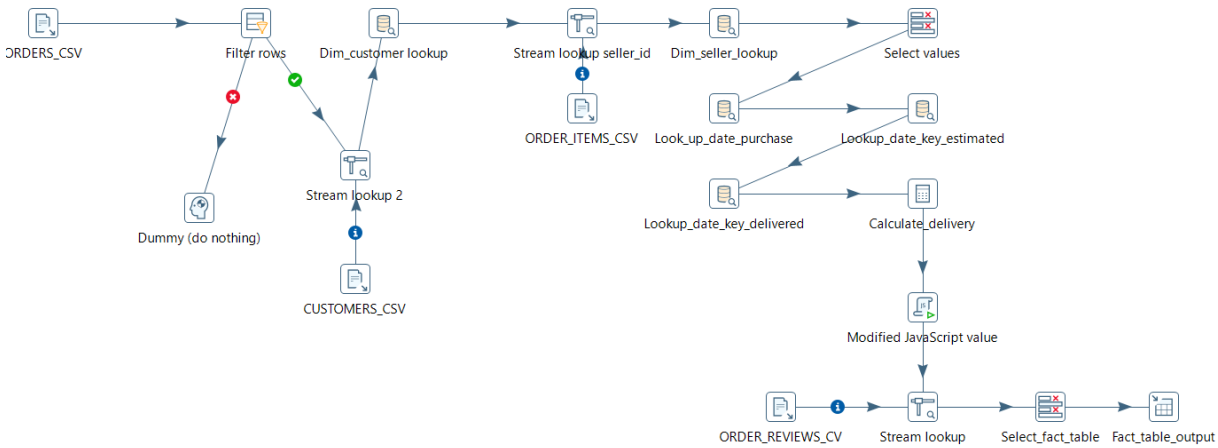
a discard stream. Category fields are then checked, and any null or empty product_category_name values are replaced with “Unknown” to maintain consistency. After basic trimming and normalization of text fields, the cleaned and validated product records are loaded into dim_product using a surrogate key during the insert/update step.

3.2.6 Transform fact_order_items



The fact_order_items table is built by joining data from olist_order_items, olist_orders, and olist_customers, bringing together product, order, and customer identifiers in a single stream. After selecting the required attributes, the pipeline trims text fields and filters out any rows where core identifiers (product_id, order_id, order_item_id, customer_unique_id, or seller_id) are null, sending invalid rows to a discard file. A JavaScript validation step cleans numeric fields, converts negative or invalid price and freight values to null, standardizes the order_date to YYYY-MM-DD format, and validates UUID based identifiers. A calculated field then computes the total price by combining item price and freight value. Once the dataset is validated, the transformation performs surrogate key lookups against dim_customer, dim_product, dim_date, and dim_seller, replacing natural keys with the corresponding dimension keys. The final cleaned and validated rows are then written to fact_order_items.

3.2.7 Transform fact_delivery



The fact_delivery table is built mostly from the Olist orders dataset, with additional lookups from customers, sellers, dates, and reviews. The pipeline begins by filtering out rows with missing order identifiers, ensuring only valid deliveries are processed. A stream lookup retrieves customer_unique_id for each order, which is then used to obtain the corresponding customer_key from

dim_customer. A second stream lookup pulls seller_id from the order items dataset so that the appropriate seller_key can be retrieved from dim_seller. Three separate database lookups assign date keys for purchase, estimated delivery, and actual delivery dates by matching these timestamps against the dim_date table. The transformation then calculates delivery_time_days and uses a JavaScript step to flag late deliveries by comparing the delivered date with the estimated delivery date. A final stream lookup attaches review data (score and comment) using order_id as the key. After selecting the required fields, the enriched and validated rows are written to fact_delivery, with all natural keys replaced by surrogate keys from the corresponding dimensions.

3.3 Data Quality Issues and Handling

During the ETL process, several data quality issues were identified in the raw Olist dataset:

- Duplicate records: Some customer and seller IDs appeared more than once. This was handled by using the Unique Rows step in Pentaho before loading dimension data.
- Missing or empty values: Fields such as city or state were sometimes blank. We used Handle Empty Fields to replace nulls with "Unknown" or exclude invalid rows where IDs were missing.
- Inconsistent text formatting: Text fields like city and state used mixed capitalization. These were standardized using the String Operations step.
- Orphan records: Orders referencing missing customers or sellers were excluded to maintain referential integrity.

These steps ensured that all keys matched between facts and dimensions, and that the warehouse contained clean and consistent data suitable for reporting.

4. Key Insights (Simple and Direct)

4.1 Sales

Sales show a clear upward trend over time. There are visible seasonal peaks, meaning some months consistently perform better. A few product categories generate most of the revenue (e.g., beleza_saude, relogios_presentes, cama_mesa_banho).

4.2 Geography

Revenue is highly concentrated in a few states, especially São Paulo (SP). Other large states (RJ, MG) perform well, but far below SP.

4.3 Deliveries

A high share of deliveries are late (around 50%).
Average delivery time is long (around 12 days).
Some regions experience more delays than others.

4.4 Customers

Despite delivery issues, the average review score is high (around 4.1).

4.5 Forecast

Forecasted sales continue to rise in the future.

Seasonal patterns are expected to repeat.

5. Business recommendations

5.1 Improve Delivery Performance

- Reduce late deliveries by reviewing logistic partners or optimizing shipping routes in high-delay regions.
- Aim to shorten the average delivery time, especially in the worst-performing states.

5.2 Focus on High-Performing Product Categories

- Allocate more marketing and stock planning to top categories like beleza_saude and relogios_presentes.
- Consider reducing or re-evaluating low-performing categories.

5.3 Strengthen Presence in Key States

- Prioritize São Paulo, Rio de Janeiro, and Minas Gerais with targeted marketing campaigns and improved shipping options to support higher demand in these regions.
- Explore potential untapped markets in mid-tier states.

5.4 Prepare for Seasonal Peaks

- Use the seasonal trend to plan inventory, staffing, and marketing ahead of high-demand months.

5.5 Maintain High Customer Satisfaction

- Since review scores are strong, maintain good communication and product quality.
- Improving delivery times will likely increase customer satisfaction even more.

6. Team member contributions

Throughout the project, all team members participated in every stage of the work. We did not divide the tasks of the project into isolated parts like “ETL only” or “Power BI only”. We did instead collaborate across the entire workflow, however some team members naturally took some more responsibility in areas they felt the strongest.

- Bjørn Erik focused primarily on the database logic, structure and overall data model, making sure relationships formed a solid base for the ETL and analytics work. Also helped with ETL pipelines.
- Lars and Anthony were mainly responsible for the ETL processes and loading the Olist datasets into the data warehouse. They also made sure the final data model was consistent and usable
- Mikkel and Anthony took the lead on designing and building the Power BI dashboards. Other group members contributed feedback and testing making sure numbers made sense.
- Andre focused on the Python scripts and predictive analytics, including the sales forecasting model and preparing results for the the project representation.

While certain members contributed more in specific areas, the final project is the result of shared teamwork where we all helped each other out with problem-solving, testing and documentation.

7. Bibliography

OpenAI. (2025). *ChatGPT (GPT-5.1)* [Large Language Model]. OpenAI https://chatgpt.com/

Anthropic. (2025). *Claude Sonnet 4.5* [Large Language Model]. Anthropic https://www.anthropic.com/claude/sonnet

Github. (2025) *Github Copilot* [AI code generation tool]. Github. https://github.com/features/copilot

Geeksforgeeks. (2025). *Sales Forecast Prediction – Python*. *https://www.geeksforgeeks.org/python/sales-forecast-prediction-python/*

Geeksforgeeks. (2025). *Data Visualization using Matplotlib in Python*. https://www.geeksforgeeks.org/data-visualization/data-visualization-using-matplotlib/

Olist (2018). Brazilian E-Commerce Public Dataset by Olist. Kaggle. André Sionek. https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce